

CyberLab — Incrementos

7 desafios extras sobre o projeto que você já fez, para praticar recursos de Next.js que a atividade base não cobriu

Como usar este material

- Cada desafio parte do projeto `cyberlab-next` que você já entregou — nenhum deles pede para recomençar do zero.
- A ordem sugerida é crescente em dificuldade: os primeiros mexem só em arquivos novos; os últimos mexem na arquitetura da página.
- Faça um de cada vez e teste no navegador antes de ir para o próximo — se algo quebrar, é mais fácil identificar o que foi.
- O bloco **Conceito** em cada desafio é o que provavelmente vai ser cobrado na prova sobre aquele tópico.

Desafio 1 — Rota dinâmica: página de detalhes **fácil**

Crie o arquivo `src/app/ferramenta/[id]/page.tsx`. Ao clicar em um card no CyberLab, o usuário deve navegar para essa rota e ver os detalhes completos daquela ferramenta.

Conceito: no App Router, uma pasta entre colchetes — `[id]` — cria um segmento de rota dinâmico. O valor da URL fica disponível no componente da página através do parâmetro `params`.

1. No `CardFerramenta.tsx`, troque o link "Ver no GitHub" (ou adicione um novo) usando `next/link`:

```
// TODO ALUNA: importar Link de "next/link"

<Link href={` /ferramenta/${id}`} className="text-xs text-emerald-400 underline">
  Ver detalhes
</Link>
```

2. Crie `src/app/ferramenta/[id]/page.tsx`:

```
// PASSO 1: tipar as props da página. No App Router, o componente
// recebe um objeto com 'params', e params.id é o valor da URL.
// Dica de tipagem: { params }: { params: { id: string } }

// PASSO 2: fazer o fetch do repositório específico usando o id
// Endereço: https://api.github.com/repositories/{id}
// (esse endpoint da API do GitHub busca um repositório pelo ID numérico)

export default async function DetalheFerramenta({ params }: /* TODO ALUNA */) {
  // PASSO 3: buscar os dados com fetch (pode usar await direto, é Server Component)
  // PASSO 4: tratar o caso de erro (repositório não encontrado)

  return (
    <main className="max-w-3xl mx-auto p-8 text-white">
      <a href="/" className="text-emerald-400 text-sm underline">- Voltar</a>
      { /* TODO ALUNA: exibir nome, descrição, estrelas e link para o GitHub */ }
    </main>
  );
}
```

Desafio 2 — Route Handler: sua própria API interna **fácil/médio**

Em vez do `page.tsx` chamar a API do GitHub diretamente, ele vai chamar uma rota sua, que por sua vez chama o GitHub.

Conceito: um arquivo `route.ts` dentro de `src/app/api/...` vira um endpoint de API que roda no seu próprio servidor Next.js — útil para esconder chaves de API, tratar erros num lugar só, ou combinar dados de várias fontes antes de enviar ao navegador.

Crie `src/app/api/ferramentas/route.ts`:

```
import { NextResponse } from "next/server";

// TODO ALUNA: criar e exportar uma função chamada GET (nome obrigatório)
// Dentro dela:
// PASSO 1: fazer o fetch para a API do GitHub (a mesma URL que você já usa)
// PASSO 2: converter a resposta em JSON
// PASSO 3: devolver os dados com NextResponse.json(...)

export async function GET() {
  // fetch("https://api.github.com/search/repositories?q=topic:cybersecurity&sort=stars&order=desc&per_page=6")
  // .then(res => res.json())
  // .then(data => ...)
}
```

No `page.tsx`, troque a URL do fetch dentro do `useEffect`:

```
// TODO ALUNA: trocar a URL do GitHub por "/api/ferramentas"
fetch("/api/ferramentas")
  .then(res => res.json())
  .then(data => /* ... */);
```

Teste: abra `http://localhost:3000/api/ferramentas` direto no navegador — deve aparecer o JSON puro, sem nenhum HTML.

Desafio 3 — Server Component vs Client Component médio

Esse é o desafio mais importante para prova: hoje seu `page.tsx` inteiro é Client Component (por causa do `"use client"`). Vamos separar o que realmente precisa rodar no navegador do que não precisa.

Conceito: por padrão, todo componente no App Router é **Server Component** (roda no servidor, pode usar `async/await` direto, não pode usar `useState` / `useEffect`). Um componente vira **Client Component** só quando você escreve `"use client"` no topo — necessário quando ele usa estado, efeitos ou interatividade (`onClick`, etc). A prática recomendada é manter Client Components pequenos e isolados, e deixar o máximo possível como Server Component.

1. Crie `src/components/ToolkitButton.tsx` (isso é Client Component — tem estado e `onClick`):

```
"use client";

import { useState } from "react";

// TODO ALUNA: criar a interface de props recebendo a lista de ferramentas
// (ou só a quantidade inicial, dependendo de como você decidir)

export default function ToolkitButton(/* TODO ALUNA: props tipadas */) {
  const [toolkit, setToolkit] = useState(0);

  return (
    <div className="bg-slate-800 px-4 py-2 rounded-lg border border-slate-700">
      Toolkit: <span className="font-bold text-emerald-400">{toolkit}</span>
      /* TODO ALUNA: você vai precisar repensar como o clique em cada
      card avisa este componente — pesquise sobre "levantar estado" (lifting state up)
      ou sobre passar uma função de callback via props para os CardFerramenta */
    </div>
  );
}
```

2. Reescreva `page.tsx` como Server Component (sem `"use client"`, sem `useState/useEffect` para os dados):

```
// TODO ALUNA: remover a linha "use client" do topo do arquivo
// TODO ALUNA: remover os imports/estados de useState e useEffect ligados a 'ferramentas' e 'loading'

async function getFerramentas() {
  // PASSO 1: fazer o fetch (pode usar await direto, sem useEffect!)
  // PASSO 2: retornar os dados já tratados
}

export default async function Home() {
  const ferramentas = await getFerramentas();

  return (
    <main className="max-w-6xl mx-auto p-8">
      <header className="mb-8 flex justify-between items-center border-b border-slate-800 pb-4">
        <div>
          <h1 className="text-3xl font-bold text-emerald-400">CyberLab</h1>
          <p className="text-slate-400 text-sm">Security Tools & Resources</p>
        </div>
        /* TODO ALUNA: usar <ToolkitButton /> aqui no lugar do contador antigo */
      </header>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        /* seu .map() de sempre, renderizando <CardFerramenta /> */
      </div>
    </main>
  );
}
```

Sem solução fechada de propósito: esse último ponto (o clique no card avisar o contador que está em outro componente) não tem um único jeito certo — é exatamente o tipo de decisão de arquitetura que separa quem entende Server/Client Components de quem só decorou a sintaxe. Pesquise "lifting state up React" e "Client Component boundary Next.js" antes de tentar.

Desafio 4 — loading.tsx e error.tsx fácil

Deixe o Next cuidar dos estados de carregamento e erro automaticamente, em vez de controlar isso manualmente com `useState`.

Conceito: arquivos com nomes especiais no App Router (`loading.tsx`, `error.tsx`, `not-found.tsx`) são convenções que o Next.js reconhece sozinho: `loading.tsx` aparece automaticamente enquanto um Server Component está buscando dados; `error.tsx` aparece se aquele componente lançar um erro.

Crie `src/app/loading.tsx`:

```
export default function Loading() {
  return (
    <div className="max-w-6xl mx-auto p-8 text-slate-400">
      Carregando ferramentas...
    </div>
  );
}
```

Crie `src/app/error.tsx` (esse precisa ser Client Component):

```
"use client";

// TODO ALUNA: tipar as props recebidas (error: Error, reset: () => void)

export default function ErrorPage(/* TODO ALUNA */) {
  return (
    <div className="max-w-6xl mx-auto p-8 text-red-400">
      Algo deu errado ao carregar as ferramentas.
      { /* TODO ALUNA: renderizar um botão que chama reset() */ }
    </div>
  );
}
```

Isso só funciona de verdade depois do Desafio 3 (page.tsx virar Server Component com fetch direto). Se seu `getFerramentas()` lançar um erro proposital (ex: `throw new Error("teste")`), o `error.tsx` deve aparecer sozinho.

Desafio 5 — Metadata dinâmica fácil

Na página de detalhes do Desafio 1, faça o título da aba do navegador mudar conforme a ferramenta.

Conceito: exportar uma função `generateMetadata` de uma `page.tsx` permite gerar o `<title>` e outras tags de SEO dinamicamente, com base nos dados da própria página (ótimo para páginas de produto, artigo, perfil, etc).

```
// Em src/app/ferramenta/[id]/page.tsx, além do componente da página:

export async function generateMetadata({ params }: /* mesma tipagem da página */) {
  // TODO ALUNA: buscar os dados da ferramenta (pode repetir o fetch do PASSO 2 do Desafio 1)
  // TODO ALUNA: retornar um objeto { title: `${nome} – CyberLab` }
}
```

Desafio 6 — Busca por query string médio

Adicione um campo de busca que filtra as ferramentas exibidas via URL, tipo `?q=nmap`.

Conceito: `useSearchParams()` lê os parâmetros da URL atual, e `useRouter()` permite mudar a URL (e portanto o estado da página) sem recarregar tudo. É a forma correta de sincronizar um filtro/busca com a URL, para que o link seja compartilhável.

Esse componente precisa ser Client Component. Uma estrutura possível — `src/components/BuscaFerramenta.tsx`:

```
"use client";

import { useRouter, useSearchParams } from "next/navigation";

export default function BuscaFerramenta() {
  const router = useRouter();
  const searchParams = useSearchParams();

  // TODO ALUNA: criar uma função que roda ao digitar no input
  // Dica: router.push(`?q=${valorDigitado}`)

  // TODO ALUNA: ler o valor atual do parâmetro 'q' com searchParams.get("q")
  // para o input já abrir preenchido se a URL já tiver ?q=algo

  return (
    <input
      type="text"
      placeholder="Buscar ferramenta..."
      className="bg-slate-800 border border-slate-700 text-white rounded-lg px-3 py-2 text-sm"
      /* TODO ALUNA: value e onChange */
    />
  );
}
```

No componente que busca os dados (page.tsx ou route.ts), você vai precisar filtrar a lista de ferramentas com base nesse `q` — pode ser um filtro simples com `.filter()` no nome, feito no servidor mesmo.

Desafio 7 — Cache e revalidate médio/avançado

Controle explicitamente por quanto tempo o Next.js guarda em cache a resposta da API do GitHub.

Conceito: quando o fetch acontece num Server Component, o Next.js pode cachear a resposta. `{ next: { revalidate: 3600 } }` diz "reaproveite essa resposta por até 1 hora antes de buscar de novo" — isso é a base do **ISR (Incremental Static Regeneration)**: páginas que parecem estáticas (rápidas), mas se atualizam sozinhas de tempos em tempos, sem você precisar reconstruir o site inteiro.

```
// No getFerramentas() do Desafio 3, ou no route.ts do Desafio 2:

const res = await fetch(
  "https://api.github.com/search/repositories?q=topic:cybersecurity&sort=stars&order=desc&per_page=6",
  // TODO ALUNA: adicionar a opção { next: { revalidate: 3600 } } aqui
);
```

Valor de fetch	Comportamento
<code>{ cache: "force-cache" }</code> (padrão)	Busca uma vez e reaproveita para sempre (até o próximo build)
<code>{ next: { revalidate: 3600 } }</code>	Reaproveita por até 1h, depois busca de novo (ISR)
<code>{ cache: "no-store" }</code>	Busca sempre, a cada requisição (sem cache)

No README do projeto, escreva 2-3 frases explicando, com suas próprias palavras, a diferença entre esse cache do servidor e o fetch feito com `useEffect` no navegador que você usou na atividade base.

Resumo — o que cada desafio cobre

#	Tópico	Onde mais aparece em prova
1	Rotas dinâmicas (<code>[id]</code>) e <code>next/link</code>	Navegação entre páginas, <code>params</code>
2	Route Handlers (<code>route.ts</code>)	Criar API própria dentro do Next.js
3	Server vs Client Components	O tópico mais cobrado em Next.js moderno
4	<code>loading.tsx</code> / <code>error.tsx</code>	Convenções de arquivo do App Router
5	<code>generateMetadata</code>	SEO dinâmico
6	<code>useSearchParams</code> / <code>useRouter</code>	Estado sincronizado com a URL
7	<code>revalidate</code> / <code>cache</code>	ISR, performance, dados "quase" estáticos